

Statistical_Modeling_Report

J.Hastings

2026-04-16

Objective: Build a Lie Detector Model

Purpose:

The goal of this project is to predict which participants will successfully complete the program based on their entrance assessment scores:

- 1 = Successfully completed the program through its duration (completer)
- 0 = Unsuccessfully completed the program prior to expiration (non-completer)

The central question driving this project is:

Can I build a model that beats the baseline accuracy of 59% derived from the Bernoulli distribution of the outcome variable?

Data Preparation: The data was cleaned to remove or correct inaccurate records.

Columns and Their Acronyms:

- **PWS** — Permanency / Navigating Welfare System
- **DL** — Daily Living
- **SC** — Self Care
- **RC** — Relationships & Communication
- **HMM** — Housing & Money Management
- **WSL** — Work & Study Life
- **CEP** — Career & Education Planning
- **LF** — Looking Forward

Model Performance Metrics

The following metrics are used to evaluate how well the model performs:

- **AUC (Area Under the Curve):** Measures how well the model separates completers from non-completers overall. A higher AUC means the model does a better job distinguishing between the two groups across different decision thresholds.
- **Accuracy:** The overall percentage of predictions the model gets correct. This shows how often the model is right, but it does not tell us whether the model performs equally well on both groups.
- **Sensitivity:** Measures how well the model correctly identifies participants who successfully complete the program. In simple terms, it shows how well the model catches positive cases.
- **Specificity:** Measures how well the model correctly identifies participants who do not complete the program. In simple terms, it shows how well the model catches negative cases.

Together, these metrics give a more complete picture of model performance than using accuracy alone.

```
# Load and format the data
```

```
my_data = read.csv("/Users/oakmoreroadinc./Desktop/Data Science /Portfolio Data Science /R_script.R/data.csv")
```

```
head(my_data)
```

```
##   Youth.ID Gender  PWS  DL  SC  RC  HMM  WSL  CEP  LF Days_in_Program
## 1         1      M 3.80 4.47 4.41 4.33 3.96 4.10 5.00 5.00             264
## 2         2      M 4.00 4.41 4.53 4.78 3.70 4.40 4.10 4.00             467
## 3         3      M 4.30 4.65 4.29 4.78 3.78 4.60 4.11 4.50              49
## 4         4      M 3.55 2.24 2.71 3.83 2.09 2.70 2.11 3.25             908
## 5         5      F 4.90 4.71 4.76 4.83 3.35 4.65 4.78 5.00             241
## 6         6      M 4.50 4.59 4.65 4.83 4.09 4.35 3.89 4.62             495
##   Category Result.Score
## 1      Good             1
## 2      Good             1
## 3      Bad              0
## 4      Good             1
## 5      Bad              0
## 6      Bad              0
```

```
glimpse(my_data)
```

```
## Rows: 22
## Columns: 13
## $ Youth.ID      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
## $ Gender        <chr> "M", "M", "M", "M", "F", "M", "M", "M", "M", "F", "M", ~
## $ PWS           <dbl> 3.80, 4.00, 4.30, 3.55, 4.90, 4.50, 3.50, 5.00, 3.00, ~
## $ DL            <dbl> 4.47, 4.41, 4.65, 2.24, 4.71, 4.59, 4.53, 4.67, 3.06, ~
## $ SC            <dbl> 4.41, 4.53, 4.29, 2.71, 4.76, 4.65, 4.82, 4.94, 3.94, ~
## $ RC            <dbl> 4.33, 4.78, 4.78, 3.83, 4.83, 4.83, 4.44, 5.00, 4.00, ~
## $ HMM           <dbl> 3.96, 3.70, 3.78, 2.09, 3.35, 4.09, 3.70, 4.40, 3.25, ~
## $ WSL           <dbl> 4.10, 4.40, 4.60, 2.70, 4.65, 4.35, 4.05, 4.84, 3.00, ~
## $ CEP           <dbl> 5.00, 4.10, 4.11, 2.11, 4.78, 3.89, 3.33, 4.78, 3.00, ~
## $ LF            <dbl> 5.00, 4.00, 4.50, 3.25, 5.00, 4.62, 4.00, 5.00, 4.00, ~
## $ Days_in_Program <int> 264, 467, 49, 908, 241, 495, 652, 184, 443, 476, 708, ~
## $ Category      <chr> "Good", "Good", "Bad", "Good", "Bad", "Bad", "Good", "Bad", ~
## $ Result.Score   <int> 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, ~
```

Baseline GAM model

```
library(dplyr)
library(mgcv)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
clean_df_gam <- my_data %>%
  select(-Youth.ID, -Category) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender),
    Avg_Metric = rowMeans(across(c(PWS, DL, SC, RC, HMM, WSL, CEP, LF)))
  ) %>%
  select(Result.Score, Gender, Avg_Metric) %>%
  as.data.frame()
```

```
n <- nrow(clean_df_gam)
pred_prob <- numeric(n)

for (i in 1:n) {
  train_data <- clean_df_gam[-i, ]
  test_data <- clean_df_gam[i, , drop = FALSE]

  baseline_model <- gam(
    Result.Score ~ Gender + s(Avg_Metric, k = 4, bs = "cs"),
    data = train_data,
    family = binomial(link = "logit"),
    method = "REML"
  )

  pred_prob[i] <- predict(baseline_model, newdata = test_data, type = "response")
}
```

Evaluate predictions on Baseline model

```
roc_obj <- roc(clean_df_gam$Result.Score, pred_prob, quiet = TRUE)
auc_value <- auc(roc_obj)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

TP <- sum(pred_class == 1 & clean_df_gam$Result.Score == 1)
TN <- sum(pred_class == 0 & clean_df_gam$Result.Score == 0)
FP <- sum(pred_class == 1 & clean_df_gam$Result.Score == 0)
FN <- sum(pred_class == 0 & clean_df_gam$Result.Score == 1)

sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
accuracy <- (TP + TN) / (TP + TN + FP + FN)

auc_value
```

```
## Area under the curve: 0.5897
```

```
sensitivity
```

```
## [1] 0.7692308
```

```
specificity
```

```
## [1] 0.4444444
```

```
accuracy
```

```
## [1] 0.6363636
```

Baseline GAM model

The baseline GAM model showed weak overall performance. Its AUC was 0.5897, meaning it performed only slightly better than random guessing when separating completers from non-completers. The model had a sensitivity of 0.7692, so it correctly identified about 77% of the participants who completed the program. However, its specificity was 0.4444, meaning it correctly identified only about 44% of the participants who did not complete the program. In simple terms, the baseline model was better at identifying completers than non-completers, but its overall predictive strength was still limited.

PCA : GAM + leak-free LOOCV

```
library(dplyr)
library(mgcv)
library(pROC)

assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")

base_df <- my_data %>%
  select(Result.Score, Gender, all_of(assess_cols)) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender)
  ) %>%
  as.data.frame()

n <- nrow(base_df)
pred_prob <- numeric(n)

for (i in 1:n) {
  train_data <- base_df[-i, ]
  test_data <- base_df[i, , drop = FALSE]

  # PCA learned on training fold only
  pca_fit <- prcomp(
    train_data[, assess_cols],
    center = TRUE,
    scale. = TRUE
```

```

)

# choose a fixed small number of PCs to avoid overfitting
train_pcs <- predict(pca_fit, newdata = train_data[, assess_cols])[, 1:2, drop = FALSE]
test_pcs <- predict(pca_fit, newdata = test_data[, assess_cols])[, 1:2, drop = FALSE]

train_fold <- data.frame(
  Result.Score = train_data$Result.Score,
  Gender = train_data$Gender,
  PC1 = train_pcs[, 1],
  PC2 = train_pcs[, 2]
)

test_fold <- data.frame(
  Gender = factor(test_data$Gender, levels = levels(train_data$Gender)),
  PC1 = test_pcs[, 1],
  PC2 = test_pcs[, 2]
)

fit_i <- gam(
  Result.Score ~ Gender + s(PC1, k = 4, bs = "cs"),
  data = train_fold,
  family = binomial(link = "logit"),
  method = "REML",
  select = TRUE
)

pred_prob[i] <- predict(fit_i, newdata = test_fold, type = "response")
}

```

```

roc_obj <- roc(base_df$Result.Score, pred_prob, quiet = TRUE)
auc_value <- auc(roc_obj)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

TP <- sum(pred_class == 1 & base_df$Result.Score == 1)
TN <- sum(pred_class == 0 & base_df$Result.Score == 0)
FP <- sum(pred_class == 1 & base_df$Result.Score == 0)
FN <- sum(pred_class == 0 & base_df$Result.Score == 1)

sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
accuracy <- (TP + TN) / (TP + TN + FP + FN)

auc_value

```

```
## Area under the curve: 0.6068
```

```
sensitivity
```

```
## [1] 0.7692308
```

```
specificity
```

```
## [1] 0.5555556
```

```
accuracy
```

```
## [1] 0.6818182
```

PCA + GAM model

The PCA-based GAM model performed somewhat better than the baseline model. Its AUC increased from 0.5897 to 0.6068, showing a modest improvement in overall class separation. Sensitivity remained about the same, while specificity improved from 0.4444 to 0.5556, meaning the model became better at identifying participants who did not complete the program. Overall, this PCA-based version was a slightly stronger and more balanced model than the baseline GAM.

LOOCV (LEAK-FREE): PC1 + GLM model

```
library(dplyr)
library(pROC)

assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")

pca_glam_df <- my_data %>%
  select(Result.Score, Gender, all_of(assess_cols)) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender)
  ) %>%
  as.data.frame()

n <- nrow(pca_glam_df)
pred_prob <- numeric(n)

for (i in 1:n) {
  train_data <- pca_glam_df[-i, ]
  test_data <- pca_glam_df[i, , drop = FALSE]

  # PCA learned on training fold only
  pca_fit <- prcomp(
    train_data[, assess_cols],
    center = TRUE,
    scale. = TRUE
  )

  # Keep only PC1
  train_pc1 <- predict(pca_fit, newdata = train_data[, assess_cols])[, 1]
  test_pc1 <- predict(pca_fit, newdata = test_data[, assess_cols])[, 1]

  train_fold <- data.frame(
```

```

    Result.Score = train_data$Result.Score,
    Gender = train_data$Gender,
    PC1 = train_pc1
  )

  test_fold <- data.frame(
    Gender = factor(test_data$Gender, levels = levels(train_data$Gender)),
    PC1 = test_pc1
  )

  glm_model <- glm(
    Result.Score ~ Gender + PC1,
    data = train_fold,
    family = binomial(link = "logit")
  )

  pred_prob[i] <- predict(glm_model, newdata = test_fold, type = "response")
}

```

```

roc_obj <- roc(base_df$Result.Score, pred_prob, quiet = TRUE)
auc_value <- auc(roc_obj)

```

```

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

```

```

TP <- sum(pred_class == 1 & base_df$Result.Score == 1)
TN <- sum(pred_class == 0 & base_df$Result.Score == 0)
FP <- sum(pred_class == 1 & base_df$Result.Score == 0)
FN <- sum(pred_class == 0 & base_df$Result.Score == 1)

```

```

sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
accuracy <- (TP + TN) / (TP + TN + FP + FN)

```

```

auc_value

```

```

## Area under the curve: 0.6154

```

```

sensitivity

```

```

## [1] 0.6923077

```

```

specificity

```

```

## [1] 0.5555556

```

```

accuracy

```

```

## [1] 0.6363636

```

GLM using PC1

The GLM model using PC1 showed modest predictive performance. It achieved an AUC of 0.6154, indicating some ability to distinguish between completers and non-completers, but the overall discrimination was still limited. The model had a sensitivity of 0.6923 and a specificity of 0.5556, showing that it performed better than random guessing but still left room for improvement. Overall, the GLM using PC1 slightly outperformed the earlier GAM models, but not by a large margin.

#SMOTE with GAM model

```
library(dplyr)
library(mgcv)
library(pROC)
library(themis)
```

```
## Loading required package: recipes
```

```
##
```

```
## Attaching package: 'recipes'
```

```
## The following object is masked from 'package:Matrix':
```

```
##
```

```
##      update
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      step
```

```
assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")
```

```
base_df <- my_data %>%
  select(Result.Score, Gender, all_of(assess_cols)) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender)
  ) %>%
  as.data.frame()
```

```
n <- nrow(base_df)
```

```
pred_prob <- numeric(n)
```

```
for (i in seq_len(n)) {
```

```
  train_data <- base_df[-i, ]
```

```
  test_data <- base_df[i, , drop = FALSE]
```

```
  # PCA learned on training fold only
```

```
  pca_fit <- prcomp(
    train_data[, assess_cols],
    center = TRUE,
    scale. = TRUE
  )
```

```

# Keep the experiment clean: use only PC1 first
train_pc1 <- predict(pca_fit, newdata = train_data[, assess_cols])[, 1]
test_pc1  <- predict(pca_fit, newdata = test_data[, assess_cols])[, 1]

train_fold <- data.frame(
  Result.Score = factor(train_data$Result.Score, levels = c(0, 1)),
  Gender = factor(train_data$Gender),
  PC1 = train_pc1
)

test_fold <- data.frame(
  Gender = factor(test_data$Gender, levels = levels(train_data$Gender)),
  PC1 = test_pc1
)

# SMOTE ONLY on the training fold
# Keep k small because your dataset is small
train_smote <- smotenc(
  train_fold,
  var = "Result.Score",
  k = 2,
  over_ratio = 1
)

# Convert back to numeric 0/1 for binomial GAM
train_smote$Result.Score <- as.integer(as.character(train_smote$Result.Score))

fit_i <- gam(
  Result.Score ~ Gender + s(PC1, k = 4, bs = "cs"),
  data = train_smote,
  family = binomial(link = "logit"),
  method = "REML",
  select = TRUE
)

pred_prob[i] <- predict(fit_i, newdata = test_fold, type = "response")
}

roc_obj <- roc(base_df$Result.Score, pred_prob, quiet = TRUE)
auc_val <- auc(roc_obj)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)
cm <- table(Predicted = pred_class, Actual = base_df$Result.Score)

roc_obj <- roc(base_df$Result.Score, pred_prob, quiet = TRUE)
auc_value <- auc(roc_obj)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

TP <- sum(pred_class == 1 & base_df$Result.Score == 1)
TN <- sum(pred_class == 0 & base_df$Result.Score == 0)
FP <- sum(pred_class == 1 & base_df$Result.Score == 0)
FN <- sum(pred_class == 0 & base_df$Result.Score == 1)

```

```
accuracy <- (TP + TN) / (TP + TN + FP + FN)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
```

```
auc_value
```

```
## Area under the curve: 0.641
```

```
sensitivity
```

```
## [1] 0.6153846
```

```
specificity
```

```
## [1] 0.6666667
```

PCA + SMOTE model

The PCA + SMOTE model achieved an AUC of 0.6154, sensitivity of 0.6153, and specificity of 0.6667. These results suggest modest overall predictive ability, with more balanced performance across the two outcome classes. While the AUC did not improve beyond the earlier PCA-based GLM, the model showed better balance between identifying completers and non-completers. This suggests that SMOTE helped improve class balance, even though it did not meaningfully improve overall class separation.

GLM + PC1 + PC2 with regularization.

```
library(dplyr)
library(glmnet)
library(pROC)

assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")

base_df <- my_data %>%
  select(Result.Score, Gender, all_of(assess_cols)) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender)
  ) %>%
  as.data.frame()

n <- nrow(base_df)
pred_prob <- numeric(n)

for (i in seq_len(n)) {
  train_data <- base_df[-i, ]
```

```

test_data <- base_df[i, , drop = FALSE]

# PCA learned on training fold only
pca_fit <- prcomp(
  train_data[, assess_cols],
  center = TRUE,
  scale. = TRUE
)

train_pcs <- predict(pca_fit, newdata = train_data[, assess_cols])[, 1:2, drop = FALSE]
test_pcs <- predict(pca_fit, newdata = test_data[, assess_cols])[, 1:2, drop = FALSE]

train_fold <- data.frame(
  Result.Score = train_data$Result.Score,
  Gender = train_data$Gender,
  PC1 = train_pcs[, 1],
  PC2 = train_pcs[, 2]
)

test_fold <- data.frame(
  Gender = factor(test_data$Gender, levels = levels(train_data$Gender)),
  PC1 = test_pcs[, 1],
  PC2 = test_pcs[, 2]
)

x_train <- model.matrix(Result.Score ~ Gender + PC1 + PC2, data = train_fold)[, -1]
y_train <- train_fold$Result.Score

x_test <- model.matrix(~ Gender + PC1 + PC2, data = test_fold)[, -1]

fit_ridge <- glmnet(
  x = x_train,
  y = y_train,
  family = "binomial",
  alpha = 0,
  lambda = 0.01
)

pred_prob[i] <- as.numeric(
  predict(fit_ridge, newx = x_test, s = 0.01, type = "response")
)
}

```

```

roc_obj <- roc(base_df$Result.Score, pred_prob, quiet = TRUE)
auc_value <- auc(roc_obj)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

TP <- sum(pred_class == 1 & base_df$Result.Score == 1)
TN <- sum(pred_class == 0 & base_df$Result.Score == 0)
FP <- sum(pred_class == 1 & base_df$Result.Score == 0)
FN <- sum(pred_class == 0 & base_df$Result.Score == 1)

```

```

accuracy <- (TP + TN) / (TP + TN + FP + FN)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)

auc_value

```

```
## Area under the curve: 0.7692
```

```
accuracy
```

```
## [1] 0.8181818
```

```
sensitivity
```

```
## [1] 0.9230769
```

```
specificity
```

```
## [1] 0.6666667
```

Regularized GLM using PC1 and PC2

The regularized GLM using PC1 and PC2 was the strongest model in this project. It achieved an AUC of 0.7692, accuracy of 0.8182, sensitivity of 0.9231, and specificity of 0.6667. This model did the best job overall at separating completers from non-completers, and it was especially strong at correctly identifying participants who completed the program. Compared with the earlier GAM and GLM models, which had AUC values around **0.59 to 0

NEW DATA PREDICTION PIPELINE using GLM (PC1,PC2 + regularization)

```

library(dplyr)
library(glmnet)

assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")

all_df <- my_data %>%
  select(Result.Score, Gender, all_of(assess_cols)) %>%
  mutate(
    Result.Score = as.integer(Result.Score),
    Gender = factor(Gender)
  ) %>%
  as.data.frame()

# Final PCA trained on ALL data
pca_final <- prcomp(
  all_df[, assess_cols],
  center = TRUE,
  scale. = TRUE

```

```

)

# Keep only PC1 and PC2
pcs_all <- predict(pca_final, newdata = all_df[, assess_cols]), 1:2, drop = FALSE]

train_final <- data.frame(
  Result.Score = all_df$Result.Score,
  Gender = all_df$Gender,
  PC1 = pcs_all[, 1],
  PC2 = pcs_all[, 2]
)

x_final <- model.matrix(Result.Score ~ Gender + PC1 + PC2, data = train_final)[-1]
y_final <- train_final$Result.Score

# Final ridge model
final_ridge <- glmnet(
  x = x_final,
  y = y_final,
  family = "binomial",
  alpha = 0,
  lambda = 0.01
)

new_input <- data.frame(
  PWS = 4.2, DL = 4.5, SC = 4.0, RC = 4.3,
  HMM = 3.9, WSL = 4.1, CEP = 4.2, LF = 4.0,
  Gender = "F"
)

# Project new data into the SAME PCA space
new_pcs <- predict(pca_final, newdata = new_input[, assess_cols]), 1:2, drop = FALSE]

new_pred_df <- data.frame(
  Gender = factor(new_input$Gender, levels = levels(all_df$Gender)),
  PC1 = new_pcs[, 1],
  PC2 = new_pcs[, 2]
)

x_new <- model.matrix(~ Gender + PC1 + PC2, data = new_pred_df)[-1]

pred_prob <- as.numeric(
  predict(final_ridge, newx = x_new, s = 0.01, type = "response")
)

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

cat("\n--- NEW DATA PREDICTION (using FINAL regularized GLM) ---\n")

##
## --- NEW DATA PREDICTION (using FINAL regularized GLM) ---

```

```
cat("Predicted probability of positive class:", round(pred_prob, 3), "\n")
```

```
## Predicted probability of positive class: 0.987
```

```
cat("Predicted class (0 = negative, 1 = positive):", pred_class, "\n")
```

```
## Predicted class (0 = negative, 1 = positive): 1
```

The new data prediction from the final regularized GLM produced a predicted probability of 0.987 for the positive class, resulting in a predicted class of 1. The model is very confident that this new participant would be classified as a successful program completer. Because the predicted probability is far above the 0.50 decision threshold, this case is not just a borderline positive prediction — it is a very strong positive prediction based on the pattern of assessment scores and gender entered into the model.

Next Steps

1. **Validate Stability** — Re-test the regularized GLM with repeated cross-validation or bootstrapping to check whether the strong results hold up across different resamples.
2. **Expand the Dataset** — Add more participant records to improve reliability and reduce the effect of small-sample variation.
3. **Test Model Simplicity** — Compare the final **PC1 + PC2 + Gender** regularized GLM against a simpler version using only **PC1 + Gender** to confirm that **PC2** is truly adding value.
4. **Deploy the Final Pipeline** — Save the final regularized GLM together with the **PCA** object so new participant data can be scored using the same transformation steps.
5. **Monitor Performance Over Time** — Track AUC, accuracy, sensitivity, and specificity as new data is added to make sure the model remains useful and stable.

Conclusion

The regularized GLM using **PC1** and **PC2** was the strongest model in this project. It achieved an AUC of 0.7692, accuracy of 0.8182, sensitivity of 0.9231, and specificity of 0.6667, outperforming the baseline GAM, the **PCA**-based GAM, the GLM using **PC1** only, and the **PCA + SMOTE** model. These results suggest that combining dimensionality reduction with regularization helped the model capture the most useful patterns in the assessment data while improving overall stability. Although the dataset is small and the findings should be interpreted with caution, the final model clearly performed best and provides the strongest foundation for future validation and deployment.